Customer Shopping Behavior Analysis

End-to-End Analytics Pipeline | Python · SQL · Power BI | 3,900+ Transactions

3,900+	18	3	4	3
Transactions	Key Columns	Loyalty Tiers	Tech Layers	Dashboard Views

SECTION 01

Project Overview & Problem Statement

Modern retail businesses collect vast amounts of transactional data but often struggle to translate raw data into actionable business intelligence. This project simulates a professional, corporate-grade data analytics workflow — from raw data ingestion to stakeholder reporting — uncovering hidden purchasing patterns, evaluating the impact of discounts and subscriptions, categorizing user segments, and providing data-driven recommendations to improve targeted marketing and customer retention.

> ***"I built an end-to-end data analytics pipeline that ingests, cleans, and analyzes over 3,900 retail transactions. Using Python for data preparation, SQL for complex querying and segmentation, and Power BI for interactive visualization, I created a comprehensive BI solution that allows stakeholders to instantly identify high-value customer segments, evaluate promotional efficiency, and monitor revenue drivers."***

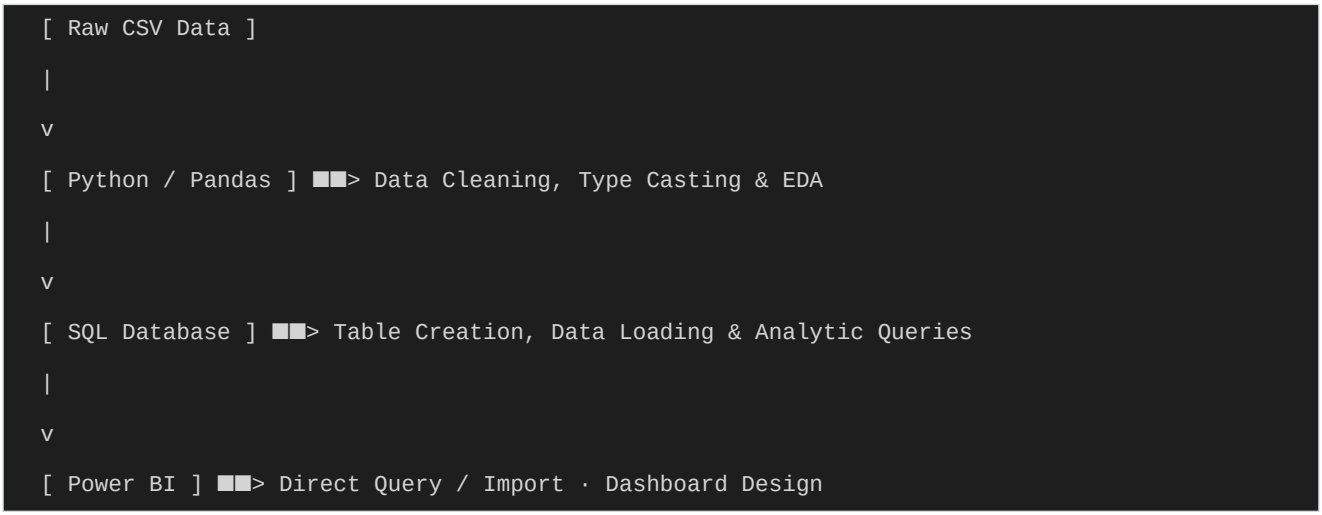
SECTION 02

Tech Stack

Layer	Technology	Why Chosen
Data Preparation	Python (Pandas), Jupyter	Ideal for fast EDA, rigorous data cleaning, and handling missing values prior to database ingestion.
Database / Querying	PostgreSQL / MySQL	Industry standard for relational analysis; supports complex aggregations, window functions, and CTEs.
Visualization / BI	Power BI	Seamless SQL integration; powerful interactive dashboarding for non-technical stakeholders.
Version Control	Git / GitHub	Maintains code history and showcases professional portfolio work.

SECTION 03

Project Architecture & Data Flow



File Structure

Customer_Shopping_Behavior_Analysis.ipynb	Python scripts for data cleaning and DB connection
customer_behavior_sql_queries.sql	Advanced SQL queries for extracting business insights
customer_behavior_dashboard.pbix	Power BI interactive dashboard file
customer_shopping_behavior.csv	The primary dataset (~3,900 records)

SECTION 04

Database / Storage Design

Data is stored in a structured relational format characterised by a single, wide **fact table** (customer) designed for analytical queries. Volume: ~3,900 records across 18 key columns.

- **Key Columns:** Customer ID, Age, Gender, Item Purchased, Category, Purchase Amount (USD), Review Rating, Subscription Status, Discount Applied, Previous Purchases, Shipping Type.
- **Architecture:** Static flat-file ELT — raw CSV is ingested via Python, cleaned, then loaded into the relational database using SQLAlchemy.

SECTION 05

Key Technical Deep-Dive: SQL Customer Segmentation

A core requirement was transforming raw purchase counts into meaningful customer cohorts. Rather than pulling raw data into Python or BI tools, this logic was pushed down to the database layer via **CTEs and CASE statements** to ensure maximum performance and reusability.

Segment	Threshold	Business Use Case
New	1 purchase	First-time buyers — target with onboarding promotions
Returning	2–10 purchases	Engaged customers — target with loyalty incentives
Loyal	>10 purchases	High-ROI cohort — target with retention and upsell campaigns

Top-3 products per category were identified using SQL **Window Functions** — partitioning by Category, ordering by purchase count descending, and assigning ROW_NUMBER(). A CTE wrapper then filtered WHERE item_rank <= 3, avoiding subquery performance pitfalls.

SECTION 06

Dashboard / UI Features

- **Revenue Overview:** High-level KPIs — Total Revenue, Average Order Value, Total Customers.
- **Demographic Breakdown:** Slicers and charts showing revenue contribution by Age Group and Gender.
- **Purchase Drivers:** Analysis of subscription status and discount impact on final purchase amounts.
- **Interactive Filtering:** Global slicers to drill into specific Product Categories (e.g., Outerwear, Footwear) or Shipping Types (Express vs. Standard).
- **Top-Down Layout:** KPIs at top → demographic breakdowns in middle → granular transaction details at bottom.

SECTION 07

Setup & Running Instructions

```
# 1. Clone the repository

git clone
https://github.com/amlanmohanty1/customer-trends-data-analysis-SQL-Python-PowerBI.git

cd customer-trends-data-analysis-SQL-Python-PowerBI

# 2. Set up Python Environment & Clean Data

# Open Customer_Shopping_Behavior_Analysis.ipynb in Jupyter

# Run all cells to clean CSV and load data into your SQL database

# 3. Execute Analytical Queries

# Open customer_behavior_sql_queries.sql in pgAdmin or DBeaver

# Run queries sequentially to extract business insights

# 4. View the Dashboard

# Open customer_behavior_dashboard.pbix in Power BI Desktop
```

SECTION 08

Challenges & Solutions

Challenge	Solution
Data Inconsistencies	Implemented a Python/Pandas preprocessing step to handle null values, standardize text formats, and enforce rigid data types before SQL ingestion.
Complex Category Rankings	Leveraged SQL Window Functions (ROW_NUMBER() OVER) to dynamically calculate the top 3 most purchased products within every category.
Bridging SQL & Power BI	Minimized Power BI model size by conducting heavy transformations and aggregations directly in SQL (Query Folding) before importing — improving refresh speed and model reusability.

SECTION 09

Limitations

- **No Time-Series Data:** The dataset lacks explicit transaction timestamps (only Season is provided), limiting month-over-month revenue trend analysis and precise cohort retention tracking.
- **Static Dataset:** Relies on a static CSV file rather than a live, streaming transactional database — requiring manual pipeline reruns for any new data.

SECTION 10

Future Improvements

- **Automated Airflow Pipeline:** Transition manual Python scripts into an Apache Airflow DAG that pulls, cleans, and loads data on a daily schedule.
- **Predictive Analytics:** Implement a Logistic Regression model (Scikit-Learn) to predict subscription purchase likelihood based on customer demographics and prior purchase history.
- **Cloud Scaling:** For 10M+ rows — migrate to a data lake (AWS S3) with a cloud warehouse (Snowflake / BigQuery) and proper indexing on frequently filtered columns.

SECTION 11

Resume Bullet Points

- Engineered an end-to-end data pipeline processing 3,900+ retail transaction records using Python (Pandas) for data cleaning, reducing raw data inconsistencies prior to database ingestion.
- Designed and executed complex SQL queries (CTEs and Window Functions) to uncover actionable business insights, segmenting customers into tiered loyalty cohorts to inform targeted marketing strategies.
- Developed an interactive Power BI dashboard serving as a central intelligence hub, visualising revenue distributions, demographic trends, and the financial impact of promotional discounts.
- Simulated a corporate data analytics workflow, translating raw transactional data into presentation-ready reports enabling non-technical stakeholders to make data-driven inventory and marketing decisions.

SECTION 12

Key Interview Q&A

Q1: Walk me through your data cleaning process.

A: I loaded the raw CSV into a Pandas DataFrame, checked for missing values, duplicated rows, and inconsistent data types — ensuring Purchase Amount was strictly a float and standardising categorical strings like Subscription Status. Once pristine, I used SQLAlchemy to load it into the relational database.

Q2: Why do aggregations in SQL rather than Power BI directly?

A: Pushing transformations to the database layer (Query Folding) reduces the Power BI model's memory footprint, speeds up refresh times, and ensures the single source of truth lives in reusable SQL logic — not proprietary DAX formulas locked inside the .pbix file.

Q3: How did you segment customers into New, Returning, and Loyal tiers?

A: I used a CTE combined with a CASE statement in SQL, evaluating the previous_purchases column: "New" for 1 purchase, "Returning" for 2–10, and "Loyal" for above 10. I then queried against the CTE to get aggregate counts per tier.

Q4: How did you identify the top 3 products in each category?

A: Using SQL Window Functions — I partitioned by Category, ordered by purchase count descending, and assigned ROW_NUMBER(). Wrapping this in a CTE let me filter the outer query with WHERE item_rank <= 3, avoiding subquery performance issues.

Q5: What was the most surprising insight in the data?

A: Comparing revenue and average spend between subscribed and non-subscribed users via SQL revealed that discount applications correlated strongly with higher-than-average order values — meaning promotional spending yielded its highest ROI on already-engaged customers, not new ones.

Q6: If this scaled to 10 million rows, what would you change?

A: Move away from local CSV loading to an Airflow-orchestrated ETL pipeline storing raw data in a data lake (AWS S3) and querying via a cloud warehouse like Snowflake or BigQuery. I'd also enforce indexing on frequently filtered columns like Category and Customer ID.

Q7: How did you design the dashboard with the end user in mind?

A: Top-down approach: the top layer surfaces high-level KPIs (Total Revenue, User Count), the middle layer shows demographic breakdowns (Gender/Age), and the bottom layer provides granular transaction detail. Global slicers let stakeholders instantly filter the entire report — e.g., Winter sales for Outerwear only.